DC MOTOR PI CONTROL DESIGN BY POLE PLACEMENT FROM AN LLS MODEL

1.0 Learning Outcomes.....1
2.0 Hardware Set-Up.....1
3.0 Practical Investigation via Simulink.....1
4.0 Concluding Remarks.....6
Recommended Citation.....6

1.0 Learning Outcomes

Upon completing this practical exercise, students should be able to demonstrate the Learning Outcomes presented in Table 1. At the end of the exercise, revisit each Learning Outcome and confirm that it has been achieved.

Table 1: *Learning Outcomes for the Practical Exercise*

ID	Learning Outcome	Achieved
1	Identify a first-order model of the ACE-Lab DC motor, including its dead-band, from a 60 s open-loop test using linear least squares (LLS).	☐
2	Calculate the gains of a PI speed controller by pole placement for a chosen damping ratio and natural frequency, and predict the closed-loop response in simulation.	☐
3	Implement the designed controllers on the hardware, verify them against the engineering requirements, and explain how the choice of design target affects the result.	☐

2.0 Hardware Set-Up

This practical exercise builds on the hardware and algorithm set-up described in the practical exercises, DC Motor Speed Control Using a PI Controller, and Parameter Estimation of a DC Motor using Linear Least Squares, which detail closed-loop speed control of the DC motor and the identification of a first-order model. The set-up is not repeated here.

As in the practical exercise, Online Parameter Estimation of a DC Motor using Recursive Least Squares, the models run on an Arduino Mega 2560 at 100 Hz in external mode (Monitor & Tune), using the same pins as the ACE-Lab (Base + Sense) (5V, GND, D2, D3 and D5). The motor is driven through the Cytron Maker Drive from a 9V battery, and every controller calculates its output in volts, limited so that no more than 6V reaches the motor (a PWM command of 0 to 170).

3.0 Practical Investigation via Simulink

MATLAB and Simulink should be used to complete this practical exercise. In the practical exercise, DC Motor PI Control: Manual Tuning, the gains were found by trial and error. Here, the gains are calculated from an identified model of the motor

by pole placement, as described in the PI Control Design by Pole Placement Learning Guide, so that every step from the measured data to the gains is traceable.

The controllers are judged against the engineering requirements in Table 2. Each requirement is checked on every reference step, and passes only if every step meets it.

Table 2: *Engineering Requirements*

ID	Engineering requirement
R1	The 10–90% rise time of each reference step shall be less than 1.0 s.
R2	The steady-state error, averaged over the last 1 s of each step, shall be no more than 0.1 RPM.
R3	The overshoot of each step shall be no more than 5% (e.g., 1 RPM on a 20 RPM step).
R4	The mean change in PWM command over the last 1 s of each step shall be no more than 5 counts per sample (no chatter).

Identify the Motor

1. Open `dc_motor_lls.slx`. It applies the same open-loop excitation as `dc_motor_rls.slx`: a PWM duty varying between 130 and 170 (4.6 V to 6 V), the sum of two square waves with periods of 3.0 s and 7.4 s.
2. Press **Monitor & Tune** and run the 60 s test.
3. Run `lls_offline.m`. It fits the first-order model with an offset, $y_k = -a_1 y_{k-1} + b_0 u_{k-1} + c$, to the logged data, sampled every $T_e = 0.05$ s, and reports the gain $K = b_0/(1+a_1)$, the time constant $\tau = -T_e/\ln(-a_1)$, the dead-band $V_{\text{dead}} = -c/b_0$ and the fit of the model.
4. Record your results in Table 3.

   Example results from an ACE-Lab motor are shown in Figure 1. The model gives $K = 15.99$ RPM/V, $\tau = 0.054$ s and a dead-band of 0.20 V, with a fit of 66.5%. The fit is limited by the quantisation of the encoder and by short dips in the speed of the gearbox, rather than by the model: the model follows every change in the measured speed.

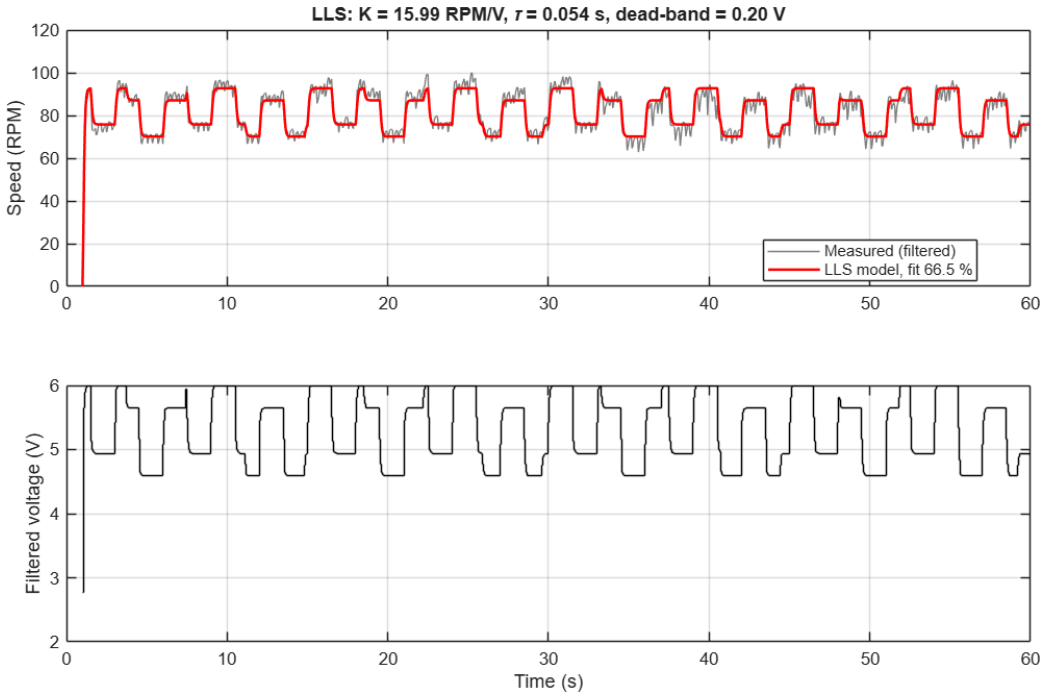


Figure 1: *LLS Identification of an ACE-Lab Motor: Measured and Simulated Speed (Top) and Filtered Voltage (Bottom)*

Table 3: *Identified Model of Your Motor*

	K (RPM/V)	τ (s)	V_{dead} (V)	Fit (%)
Example ACE-Lab motor	15.99	0.054	0.20	66.5
Your motor				

Design the PI Controller by Pole Placement

The identified model is converted to the control interval, T_s = 0.01 s, and the PI gains are calculated from Equation (10) of the PI Control Design by Pole Placement Learning Guide:

$$K_p = \frac{p - p_2}{b_s}, \qquad K_i = \frac{1 + p_1 + p_2}{b_s T_s}, \qquad p = e^{-T_s/\tau}, \qquad b_s = K(1 - p) \quad (1)$$

where $p_1 = -2e^{-\zeta\omega nTs}$ and $p_2 = e^{-2\zeta\omega nTs}$ for critical damping, ζ = 1. The script below calculates the gains for a list of natural frequencies, and simulates the predicted response of the identified model to a 20 RPM step, including the 0–6 V limit on the controller output.

MATLAB Script: pole-placement PI design from the identified model

```
K = 15.99;  tau = 0.054;          % your identified model (Table 3)
Ts = 0.01;  zeta = 1;  Vs = 6;    % sampling interval [s], limit [V]
wn_list = [6, 1/tau, 30];         % natural frequencies [rad/s]

p = exp(-Ts/tau);  bs = K*(1 - p);
N = 150;  t = (0:N-1)'*Ts;  r = 20*ones(N,1);   % 20 RPM step
figure; hold on; grid on;
for wn = wn_list
    p1 = -2*exp(-zeta*wn*Ts);  p2 = exp(-2*zeta*wn*Ts);  % zeta = 1
    Kp = max((p - p2)/bs, 0);  Ki = max((1 + p1 + p2)/(bs*Ts), 0);
    % simulate the velocity-form PI controller and the motor model
    y = zeros(N,1);  u = 0;  ePrev = 0;
    for k = 1:N-1
        e = r(k) - y(k);
```

```
        u = min(max(u + (Kp + Ki*Ts)*e - Kp*ePrev, 0), Vs);
        ePrev = e;
        y(k+1) = p*y(k) + bs*u;
    end
    k10 = find(y >= 2, 1);  k90 = find(y >= 18, 1);
    fprintf('wn = %5.1f: Kp = %.4f, Ki = %.3f, rise %.2f s, OS %.1f %%\n', ...
            wn, Kp, Ki, (k90 - k10)*Ts, max(0, max(y)/20 - 1)*100);
    plot(t, y);
end
xlabel('Time (s)'); ylabel('Speed change (RPM)');
legend(compose('\\omega_n = %.1f rad/s', wn_list), 'Location', 'southeast');
```

1. Enter your identified K and τ, and run the script. Record the gains and the predicted rise time and overshoot in Table 4.

2. Check the condition $2\zeta\omega_n\tau > 1$ for each design. For which value of ω_n is the proportional gain clipped to zero, and what does this mean for the controller?

   For the example motor, ω_n = 6 rad/s gives $2\zeta\omega_n\tau$ = 0.65 < 1, so K_p is clipped to zero and the controller is integral-only, with K_i = 0.125 V/(RPM s). This design target is slower than the motor itself, and the predicted rise time is about one second. A target between $1/\tau$ and $5/\tau$, here about 18 to 90 rad/s, gives a positive proportional gain and a much faster response: ω_n = 30 rad/s gives K_p = 0.104 V/RPM and K_i = 2.48 V/(RPM s).

Implement and Verify the Controllers on the Hardware

The controllers are tested in `dc_motor_str.slx`, the self-tuning PI model used in later practical exercises, with adaptation switched off, so that it runs a fixed PI controller with the gains you specify. The speed reference alternates between 70 and 90 RPM every 3 s, above the sticking region of the motor, and each run lasts 100 s. The model also contains a software load, which is used in the practical exercise, DC Motor Self-Tuning PI Control using Recursive Least Squares; here it is switched off by setting its factor to 1.

1. Open `dc_motor_str.slx` and set up the first design, using your gains for ω_n = 6 rad/s:

MATLAB Script: setting up dc_motor_str.slx as a fixed PI controller

```
m = 'dc_motor_str';  load_system(m);
mws = get_param(m, 'ModelWorkspace');
assignin(mws, 'adapt_on', 0);          % fixed gains: adaptation off
assignin(mws, 'Kp0', Kp);              % your designed gains
assignin(mws, 'Ki0', Ki);
assignin(mws, 'K_factor', 1);          % no software load in this exercise
assignin('base', 'hw_method', 'PP wn 6');   % name of this run
assignin('base', 'hw_scenario', 'A');
open_system(m);
```

2. Press **Monitor & Tune** and wait for the 100 s run to finish.

3. Run `res = hw_collect();`. This saves the logged data, measures the rise time, overshoot, steady-state error and chatter of every reference step, prints the worst case for the steps before and after t = 40 s, with PASS or FAIL for R1 to R4, and plots the speed, controller output and gains.

4. Calculate the median performance over the steps from 10 s to 37 s, using the per-step measures in `res.P` (columns: step time, reference, rise time, overshoot, steady-state error, chatter):

MATLAB Script: median performance over the steps from 10 s to 37 s

```
P = res.P;  sel = P(:,1) >= 10 & P(:,1) < 37;
fprintf('Median rise %.2f s, overshoot %.1f %%, chatter %.1f counts\n', ...
        median(P(sel,3)), median(P(sel,4)), median(P(sel,6)));
ok = [P(sel,3) < 1, P(sel,5) <= 0.1, P(sel,4) <= 5, P(sel,6) <= 5];
fprintf('R1 to R4 met on %.0f, %.0f, %.0f, %.0f %% of the steps\n', ...
        100*mean(ok));
```

5. Repeat for $\omega_n = 1/\tau$ and $\omega_n = 30$ rad/s, changing `hw_method` each time, and record your results in Table 4.

Table 4: *Pole-Placement Designs for Your Motor: Predicted and Measured Performance*

ω_n (rad/s)	K_p (V/RPM)	K_i (V/(RPM s))	Predicted rise time (s)	Measured median rise time (s)	Measured median overshoot (%)	Requirements met (R1–R4)
6						
$1/\tau$ =						
30						

Example hardware results for the design with $\omega_n = 6$ rad/s are shown in Figure 2. The proportional gain is zero and the integral gain is 0.125 V/(RPM s), so the controller is integral-only. Before the software load is applied at 40 s, the median rise time was 0.61 s and the median overshoot 20%, measured on speed smoothed over 0.15 s (`movmean(y, 15)`); measured on the filtered speed alone, as in `res.P`, the overshoot appears larger, because each encoder count adds 1.68 RPM of quantisation. The controller met R1 and R4 on every step, but met R2 on only 22% of the steps and R3 on none of them. The model was accurate; the controller was slow and overshot because the design target asked for a loop slower than the motor.

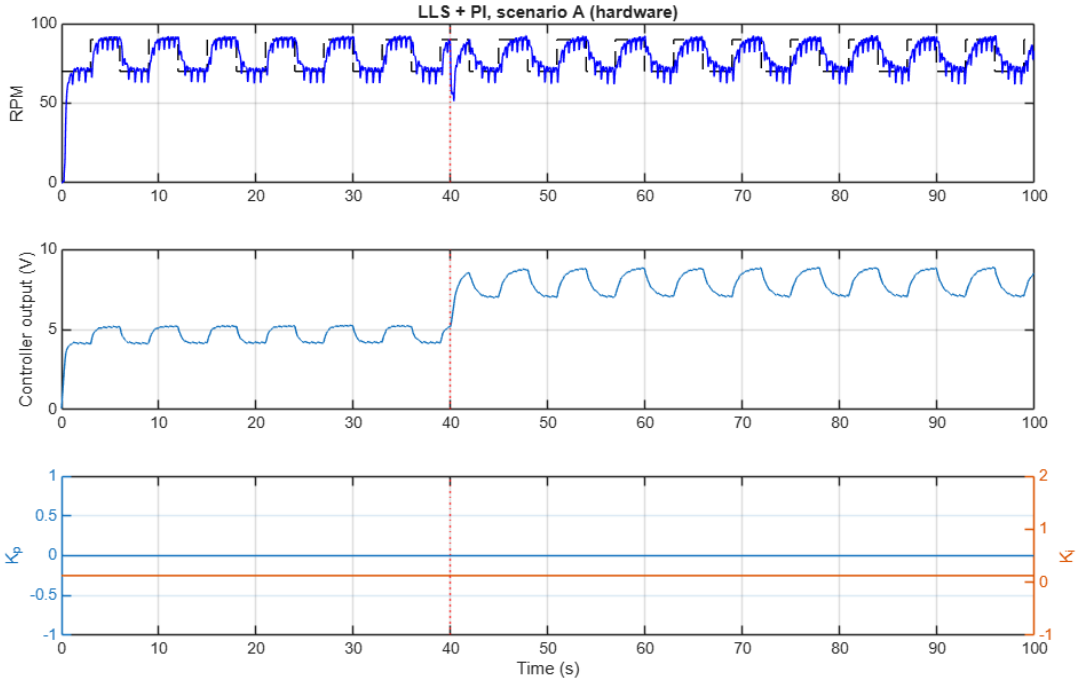


Figure 2: *Example Hardware Results for the Pole-Placement PI Controller with $\omega_n = 6$ rad/s: Speed (Top), Controller Output (Middle) and Gains (Bottom). In this Run a Software Load Was Applied at t = 40 s*

Compare the measured and predicted performance of your three designs, and explain any differences. In your explanation, consider the following:

- One encoder count per 0.01 s sample is about 1.68 RPM. What does this mean for the 0.1 RPM limit in R2?
- When a step fails R3, does the overshoot come from the controller, or from a short mechanical dip in the speed of the gearbox?
- Which design would you choose for this motor, and why? Does it meet R4 as well as R1 to R3?

4.0 Concluding Remarks

This practical exercise designed a PI speed controller for the ACE-Lab DC motor directly from an identified model. Using MATLAB, Simulink and ACE-Lab hardware, linear least squares identified the gain, time constant and dead-band of the motor from a 60 s open-loop test, and pole placement converted the model and a chosen damping ratio and natural frequency into PI gains. Each design was simulated before it was deployed to the hardware and verified against the engineering requirements.

This practical exercise also showed that the design target matters as much as the model. A target slower than the motor itself, $2\zeta\omega_n\tau < 1$, forces the proportional gain to zero and gives a slow, integral-only controller, however accurate the model. A target between $1/\tau$ and $5/\tau$ gives a faster response, while the limits of the hardware, such as the quantisation of the encoder, determine which requirements any controller can meet. The controller designed here has fixed gains: the practical exercise, DC Motor Self-Tuning PI Control using Recursive Least Squares, repeats the design automatically whenever the motor changes.

Recommended Citation

If you have used this practical exercise in your teaching, learning or research, please cite it using the reference below, adapting the format as required by your preferred referencing style.

Pickering, J. E. (2026). DC Motor PI Control Design by Pole Placement from an LLS Model. ACE-Lab. Teaching Notes: ACE-Lab Practical Exercises, Version 1.0. Zenodo. https://doi.org/10.5281/zenodo.23671552